From Matchings to Max-Flow Min-Cut

Daniel Mortensen

April 10, 2026

Contents

1 Matchings and Alternating Paths **3**
1.1 Bipartite Graphs . . . 3
1.2 Matchings . . . 3
1.3 Alternating and Augmenting Paths . . . 3
1.4 Algorithm . . . 3
1.5 Example: Augmenting Paths Step by Step . . . 3

2 From Matchings to Network Flow **5**
2.1 Adding a Source and Sink . . . 5
2.2 Ford–Fulkerson on the Same Graph . . . 5
2.3 Applications . . . 7

3 Beyond Bipartite Graphs **7**
3.1 Every Cut Is a Bipartite Interface . . . 8
3.2 Internal Edges Are Lossless Plumbing . . . 8
3.3 Nested Cuts Capture Internal Limits . . . 8

4 Formal Definitions: Flows and Cuts **9**
4.1 Flow Networks . . . 9
4.2 Flows . . . 9
4.3 Cuts . . . 9

5 The Max-Flow Min-Cut Theorem **10**
5.1 Statement . . . 10
5.2 Residual Graph . . . 10
5.3 Proof of Weak Duality . . . 10
5.4 Proof of Strong Duality . . . 10

6 Algorithm and Examples **11**
6.1 Ford–Fulkerson . . . 11
6.2 Example . . . 11

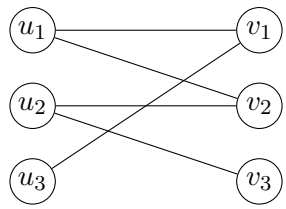
7 Summary and Related Results **11**
7.1 Hall's Theorem . . . 11
7.2 Birkhoff–von Neumann Theorem . . . 11
7.3 König–Egerváry Theorem . . . 12

7.4 Menger's Theorem 12
7.5 Unifying View 12

1 Matchings and Alternating Paths

1.1 Bipartite Graphs

A **bipartite graph** $G = (U \cup V, E)$ has two disjoint vertex sets with edges only between them.



The fundamental question is: *how many independent pairs can we match?*

1.2 Matchings

A **matching** $M \subseteq E$ is a set of edges with no shared endpoints. A vertex is **matched** if it belongs to some edge in M; otherwise it is **free**. A **maximum matching** maximises $|M|$. A **perfect matching** matches every vertex ($|M| = |U| = |V|$).

1.3 Alternating and Augmenting Paths

Given a matching M, an **alternating path** alternates between edges in M and edges not in M. An **augmenting path** is an alternating path whose two endpoints are both free.

If an augmenting path exists, flipping every edge along it (matched $\leftrightarrow$ unmatched) yields a matching of size $|M| + 1$.

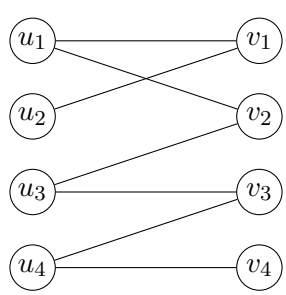
Theorem 1 (Berge, 1957)**.** *A matching M is maximum if and only if there is no augmenting path with respect to M.*

1.4 Algorithm

Berge's theorem gives an algorithm: start with $M = \emptyset$, repeatedly search for an augmenting path P (via BFS or DFS from free vertices), and set $M \leftarrow M \oplus P$ (symmetric difference). Stop when no augmenting path exists. Each augmentation increases $|M|$ by one, so the algorithm terminates after at most $\lfloor |V|/2 \rfloor$ iterations.

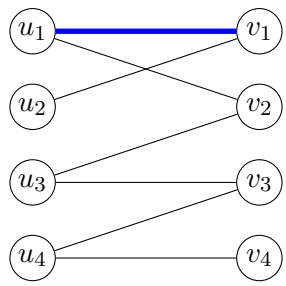
1.5 Example: Augmenting Paths Step by Step

Consider the bipartite graph with $U = \{u_1, u_2, u_3, u_4\}$, $V = \{v_1, v_2, v_3, v_4\}$ and edges:



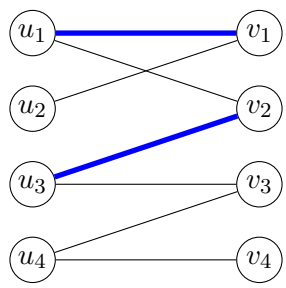
Note that u_2 has only one neighbour (v_1), so any perfect matching must assign $u_2 \to v_1$. The algorithm does not know this in advance.

Step 1. $M = \emptyset$. Search from u_1 (free): find v_1 (free). Augmenting path u_1–v_1. Augment.



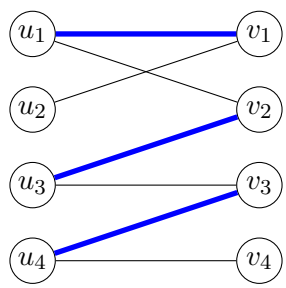
$M = \{(u_1, v_1)\}$, $|M| = 1$.

Step 2. Search from u_3 (free): find v_2 (free). Augmenting path u_3–v_2. Augment.



$M = \{(u_1, v_1), (u_3, v_2)\}$, $|M| = 2$.

Step 3. Search from u_4 (free): find v_3 (free). Augmenting path u_4–v_3. Augment.

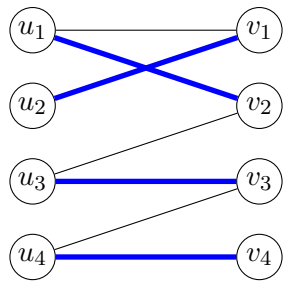


$M = \{(u_1, v_1), (u_3, v_2), (u_4, v_3)\}$, $|M| = 3$.

Step 4. The only free vertex in U is u_2. Its sole neighbour is v_1, which is matched to u_1. No free vertex is directly reachable, so BFS must follow alternating edges through the matching:

$$\underset{\text{free}}{u_2} \xrightarrow{\text{unmatched}} v_1 \xrightarrow{\text{matched}} u_1 \xrightarrow{\text{unmatched}} v_2 \xrightarrow{\text{matched}} u_3 \xrightarrow{\text{unmatched}} v_3 \xrightarrow{\text{matched}} u_4 \xrightarrow{\text{unmatched}} \underset{\text{free}}{v_4}$$

This is an augmenting path of length 7 (four unmatched edges, three matched edges). Flipping every edge along the path yields:



$M = \{(u_2, v_1), (u_1, v_2), (u_3, v_3), (u_4, v_4)\}$, $|M| = 4$. This is a perfect matching.

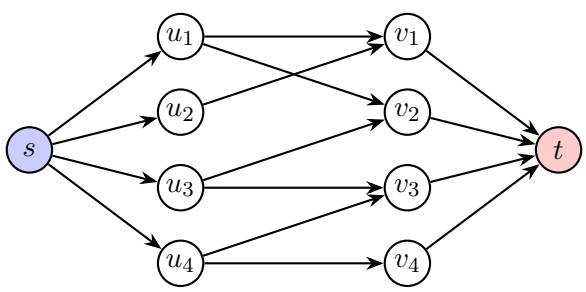
Step 5. No free vertices remain. No augmenting path exists. By Berge's theorem, M is a maximum matching.

The key observation is Step 4: the algorithm rerouted three existing matched edges through a single augmenting path. The "cascade" $v_1 \to u_1 \to v_2 \to u_3 \to v_3 \to u_4$ displaced every previous assignment to make room for u_2. Without this ability to undo prior choices, a greedy algorithm would be stuck at $|M| = 3$.

2 From Matchings to Network Flow

2.1 Adding a Source and Sink

A bipartite matching problem is a network flow problem in disguise. We convert the same graph from Section 1 into a flow network by adding a source s with capacity-1 edges to every $u \in U$, a sink t with capacity-1 edges from every $v \in V$, and directing the original edges from U to V with capacity 1. All capacities are 1.



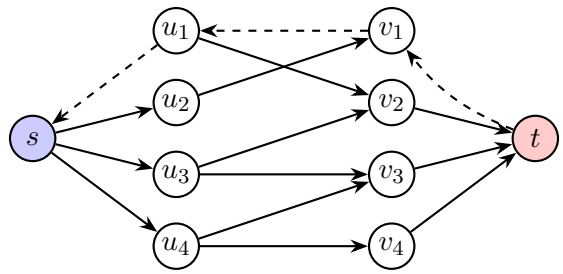
An integer maximum flow in this network corresponds exactly to a maximum matching: each unit of flow along $s \to u_i \to v_j \to t$ represents the matched edge (u_i, v_j).

2.2 Ford–Fulkerson on the Same Graph

We now trace the Ford–Fulkerson algorithm on this network. At each step we show the residual graph, find an augmenting $s \to t$ path, and push one unit of flow. Edges labelled with their residual capacity; forward edges are solid, backward edges are dashed.

Iteration 1. All flows are zero. The residual graph equals the original network. BFS from s finds the path $s \to u_1 \to v_1 \to t$ with bottleneck $\delta = 1$. Push 1 unit.

Flow: $f(s, u_1) = f(u_1, v_1) = f(v_1, t) = 1$. Total $|f| = 1$.



Note the backward edges $u_1 \dashleftarrow v_1$, $s \dashleftarrow u_1$, and $v_1 \dashleftarrow t$: these allow future iterations to undo this flow.

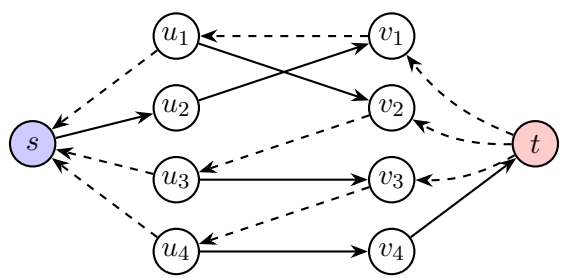
Iteration 2. BFS from s finds $s \to u_3 \to v_2 \to t$. Push 1 unit.

Flow: additionally $f(s, u_3) = f(u_3, v_2) = f(v_2, t) = 1$. Total $|f| = 2$.

Iteration 3. BFS from s finds $s \to u_4 \to v_3 \to t$. Push 1 unit.

Flow: additionally $f(s, u_4) = f(u_4, v_3) = f(v_3, t) = 1$. Total $|f| = 3$.

The residual graph after three iterations:



Iteration 4. BFS from s. The only forward edge from s is $s \to u_2$. From u_2, the only forward edge is $u_2 \to v_1$. But $v_1 \to t$ is saturated (only the backward edge $t \to v_1$ exists). So BFS must use the backward edge $v_1 \to u_1$ (undoing the flow $u_1 \to v_1$), then the forward edge $u_1 \to v_2$, then backward $v_2 \to u_3$, then forward $u_3 \to v_3$, then backward $v_3 \to u_4$, then forward $u_4 \to v_4$, then forward $v_4 \to t$.

The augmenting path in the residual graph:

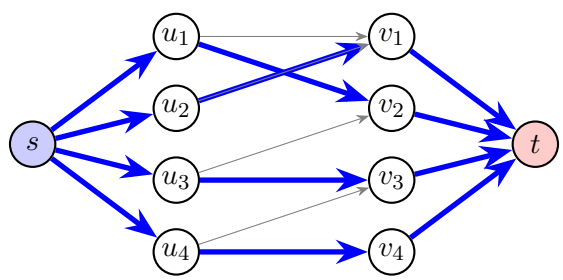
$$s \to u_2 \to v_1 \dashrightarrow u_1 \to v_2 \dashrightarrow u_3 \to v_3 \dashrightarrow u_4 \to v_4 \to t$$

where $\to$ denotes a forward edge and $\dashrightarrow$ a backward edge (undoing existing flow).

Bottleneck $\delta = 1$ (all residual capacities are 1). Push 1 unit along this path. For each forward edge, increase flow by 1; for each backward edge, *decrease* flow by 1 on the reverse arc:

Edge in path	Type	Update
$s \to u_2$	forward	$f(s, u_2) : 0 \to 1$
$u_2 \to v_1$	forward	$f(u_2, v_1) : 0 \to 1$
$v_1 \to u_1$	backward	$f(u_1, v_1) : 1 \to 0$
$u_1 \to v_2$	forward	$f(u_1, v_2) : 0 \to 1$
$v_2 \to u_3$	backward	$f(u_3, v_2) : 1 \to 0$
$u_3 \to v_3$	forward	$f(u_3, v_3) : 0 \to 1$
$v_3 \to u_4$	backward	$f(u_4, v_3) : 1 \to 0$
$u_4 \to v_4$	forward	$f(u_4, v_4) : 0 \to 1$
$v_4 \to t$	forward	$f(v_4, t) : 0 \to 1$

Total $|f| = 4$. The resulting flow (and corresponding matching):



Iteration 5. In the residual graph, all edges from s are saturated (backward only). No $s \to t$ path exists. The algorithm terminates.

The set of vertices reachable from s in the final residual graph is $S^* = \{s\}$ (no forward edge leaves s). The minimum cut is $(\{s\}, V \setminus \{s\})$ with capacity 4, confirming $\max |f| = \min c(S, T) = 4$.

Comparison with Section 1. The flow in iteration 4 corresponds exactly to the augmenting path in Step 4 of the matching algorithm. The backward edges in the residual graph play the same role as the matched edges in the alternating path: they let the algorithm *undo* previous flow assignments. Ford–Fulkerson on the flow network is the augmenting-paths algorithm in disguise.

2.3 Applications

Network flow models appear throughout combinatorics and applied mathematics: bipartite matching (job assignment, scheduling), transportation and logistics, telecommunications routing, image segmentation (min-cut separates foreground from background), baseball elimination, and—for our purposes—assigning components to lines.

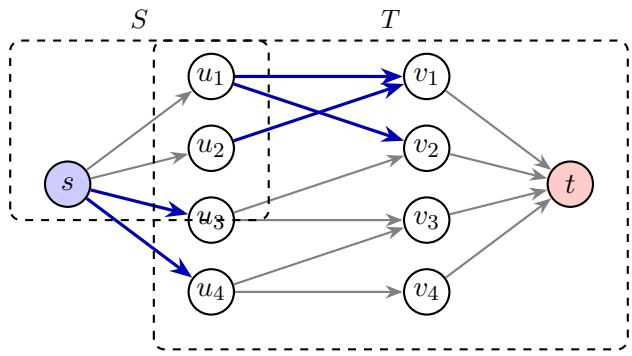
3 Beyond Bipartite Graphs

The bipartite flow network above has a simple layered structure, but flow and cuts apply to arbitrary directed graphs. The reason is that every cut in a general graph *already is* a bipartite bottleneck.

3.1 Every Cut Is a Bipartite Interface

Take any s-t cut (S, T) in a directed graph. The edges crossing from S to T form a bipartite structure: each has one endpoint in S and one in T. Every unit of flow from s to t must cross this interface.

Consider the cut $S = \{s, u_1, u_2\}$, $T = \{u_3, u_4, v_1, v_2, v_3, v_4, t\}$ in our flow network:



The five bold edges are the bipartite crossing (capacity 5); the gray edges are internal plumbing. No matter what happens inside S or T, flow from s to t cannot exceed 5 through this cut.

3.2 Internal Edges Are Lossless Plumbing

Flow conservation forces every internal node to output exactly what it receives. The subgraph inside S merely *routes* flow from s to the cut boundary—it cannot create or destroy flow. Symmetrically, the subgraph inside T routes flow from the boundary to t.

In our example, u_1 and u_2 sit inside S. They receive flow from s and pass all of it across the cut to v_1 and v_2. They are plumbing, not bottlenecks.

Therefore, for any cut (S, T):

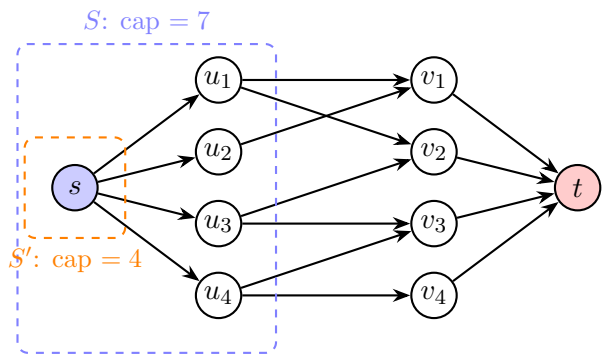
$$|f| = \underbrace{\sum_{u \in S,\, v \in T} f(u, v)}_{\text{flow across the cut}} - \underbrace{\sum_{u \in T,\, v \in S} f(u, v)}_{\text{flow back across}} \leq c(S, T).$$

The bottleneck is always at the bipartite crossing, never inside either side.

3.3 Nested Cuts Capture Internal Limits

An immediate objection: what if the edges *inside* S are too narrow to deliver flow to the cut boundary? Then a tighter cut exists that captures the true bottleneck.

Compare two cuts in our flow network:



The wider cut $S = \{s, u_1, u_2, u_3, u_4\}$ has 7 edges crossing to V (capacity 7). But s has only 4 outgoing edges, so the internal plumbing cannot deliver more than 4 to the boundary. The tighter cut $S' = \{s\}$ exposes the real bottleneck: capacity 4.

Internal limitations never hide from cuts—they just mean the minimum cut is closer to s (or t). Every general graph is a chain of bipartite bottlenecks; the minimum cut finds the tightest one.

4 Formal Definitions: Flows and Cuts

We now make the preceding intuition precise.

4.1 Flow Networks

Definition 1 (Network)**.** *A **network** is a directed graph $G = (V, E)$ with a non-negative capacity function $c : E \to \mathbb{R}_{\geq 0}$ and no multiple arcs. Without loss of generality, if $(u, v) \in E$ then $(v, u) \in E$ as well (add it with $c(v, u) = 0$ if absent).*

Definition 2 (Flow network)**.** *If two vertices are distinguished—a **source** s and a **sink** t—then (G, c, s, t) is a **flow network**.*

4.2 Flows

Definition 3 (Flow)**.** *A **flow** is a function $f : E \to \mathbb{R}_{\geq 0}$ satisfying:*

1. ***Capacity**: $0 \leq f(u, v) \leq c(u, v)$ for all $(u, v) \in E$.*
2. ***Conservation**: for every $v \in V \setminus \{s, t\}$,*

$$\sum_{u:(u,v)\in E} f(u, v) = \sum_{w:(v,w)\in E} f(v, w).$$

The **value** of a flow is $|f| = \sum_{v:(s,v)\in E} f(s, v) - \sum_{u:(u,s)\in E} f(u, s)$.

4.3 Cuts

Definition 4 (s-t cut)**.** *An s-t **cut** (S, T) is a partition $V = S \cup T$ with $s \in S$ and $t \in T$. Its **capacity** is*

$$c(S, T) = \sum_{\substack{u\in S,\, v\in T \\ (u,v)\in E}} c(u, v).$$

A cut is a bottleneck: it bounds the maximum flow that can cross from the s-side to the t-side.

5 The Max-Flow Min-Cut Theorem

5.1 Statement

Theorem 2 (Ford–Fulkerson, 1956)**.** *In any flow network* (G, c, s, t)*,*

$$\max_f |f| = \min_{(S,T)} c(S,T).$$

The following are equivalent: (i) f is a maximum flow; (ii) there is no augmenting path in the residual graph G_f; (iii) $|f| = c(S,T)$ for some cut (S,T).

5.2 Residual Graph

Given flow f, the **residual graph** $G_f = (V, E_f)$ contains a **forward edge** (u,v) with capacity $c_f(u,v) = c(u,v) - f(u,v)$ whenever this is positive, and a **backward edge** (v,u) with capacity $c_f(v,u) = f(u,v)$ whenever $f(u,v) > 0$. An **augmenting path** is any $s \to t$ path in G_f. Backward edges allow the algorithm to undo previously pushed flow—the same mechanism as augmenting paths in matching.

5.3 Proof of Weak Duality

Claim. For any flow f and any cut (S,T), $|f| \le c(S,T)$.

Proof. By flow conservation the net flow leaving S equals $|f|$:

$$|f| = \sum_{u \in S,\, v \in T} f(u,v) - \sum_{u \in T,\, v \in S} f(u,v) \le \sum_{u \in S,\, v \in T} f(u,v) \le \sum_{u \in S,\, v \in T} c(u,v) = c(S,T).$$

The first inequality drops a non-negative term; the second uses the capacity constraint. □

5.4 Proof of Strong Duality

Claim. There exist f^* and (S^*, T^*) with $|f^*| = c(S^*, T^*)$.

Proof. Run Ford–Fulkerson until no augmenting path exists; call the resulting flow f^*. Define

$$S^* = \{v \in V : v \text{ reachable from } s \text{ in } G_{f^*}\}, \qquad T^* = V \setminus S^*.$$

Since $t \notin S^*$, this is a valid cut. For every edge (u,v) with $u \in S^*$, $v \in T^*$ we have $f^*(u,v) = c(u,v)$—otherwise v would be reachable via a forward edge. For every edge (v,u) with $v \in T^*$, $u \in S^*$ we have $f^*(v,u) = 0$—otherwise v would be reachable via a backward edge. Therefore

$$|f^*| = \sum_{u \in S^*,\, v \in T^*} f^*(u,v) - \sum_{v \in T^*,\, u \in S^*} f^*(v,u) = c(S^*, T^*) - 0 = c(S^*, T^*).$$

By weak duality f^* is maximum and (S^*, T^*) is minimum. □

6 Algorithm and Examples

6.1 Ford–Fulkerson

Input: Flow network (G, c, s, t).
Output: Maximum flow f.

1. Set $f(u, v) \leftarrow 0$ for all $(u, v) \in E$.
2. While there exists an augmenting path P in G_f:
   (a) Let $\delta = \min_{(u,v)\in P} c_f(u, v)$.
   (b) For each forward edge $(u, v) \in P$, set $f(u, v) \leftarrow f(u, v) + \delta$.
   (c) For each backward edge $(v, u) \in P$, set $f(v, u) \leftarrow f(v, u) - \delta$.
3. Return f.

The **Edmonds–Karp** variant uses BFS to find the shortest augmenting path, giving worst-case complexity $O(VE^2)$.

6.2 Example

We trace the algorithm on a small network: start with zero flow, find an augmenting s-t path in the residual graph, push flow along it, update the residual graph (forward edges shrink, backward edges appear), and repeat. When no path exists, the vertices reachable from s in the final residual graph form S^*; the cut $(S^*, V \setminus S^*)$ is the minimum cut.

7 Summary and Related Results

The max-flow min-cut theorem is the cornerstone of a family of duality results in combinatorics.

7.1 Hall's Theorem

Theorem 3 (Hall, 1935)**.** *A bipartite graph $G = (U \cup V, E)$ has a matching saturating every vertex in U iff $|N(S)| \geq |S|$ for all $S \subseteq U$.*

Hall's condition is equivalent to every s-t cut having capacity at least $|U|$ in the corresponding flow network.

7.2 Birkhoff–von Neumann Theorem

Theorem 4 (Birkhoff, 1946; von Neumann, 1953)**.** *Every doubly stochastic matrix is a convex combination of permutation matrices.*

A doubly stochastic matrix has non-negative entries with rows and columns summing to 1. The theorem says the LP relaxation of bipartite matching has no integrality gap: any fractional matching decomposes into integer matchings.

7.3 König–Egerváry Theorem

Theorem 5 (König, 1931; Egerváry, 1931)**.** *In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.*

A vertex cover is a set of vertices touching every edge. The minimum vertex cover corresponds to a minimum cut in the flow network.

7.4 Menger's Theorem

Theorem 6 (Menger, 1927)**.** *The maximum number of vertex-disjoint s-t paths equals the minimum number of vertices (other than s, t) whose removal disconnects s from t.*

The edge-disjoint version—max edge-disjoint paths equals min edge cut—is a direct corollary of max-flow min-cut with unit capacities.

7.5 Unifying View

Result	**Says**
Max-flow min-cut	max throughput = min bottleneck
Hall	perfect matching $\iff$ no small neighbourhood
Birkhoff–von Neumann	fractional matching = integer matching
König–Egerváry	max matching = min vertex cover
Menger	max disjoint paths = min separator

All are facets of the same duality: **what you can push through a network equals what it takes to block it**.